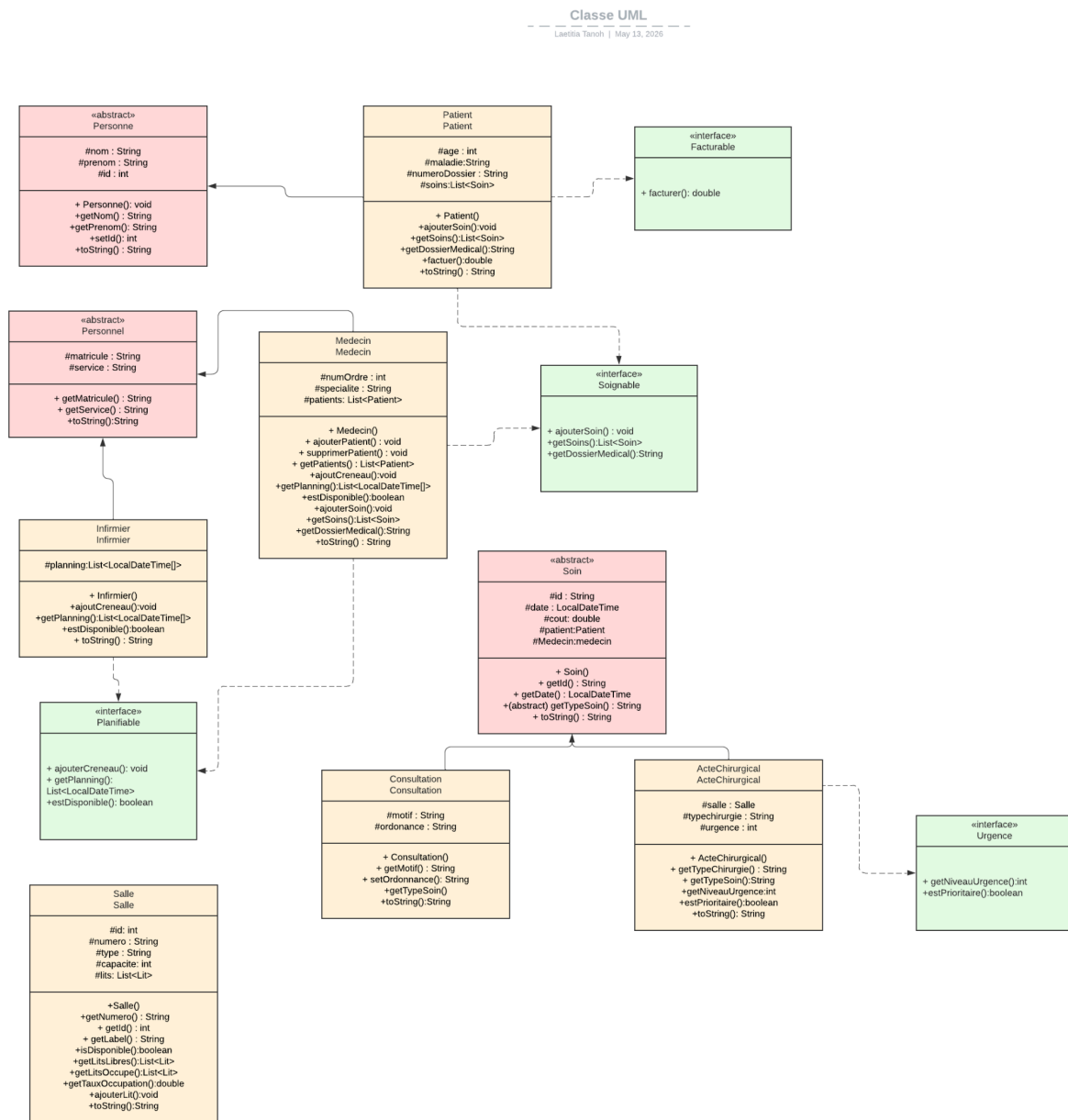


PROJET DE GROUPE - POO AVANCÉE

Réalisé par VOUSEMER Doryan, TANOI Laetitia, LOKO Marie-Paule et NICAUD Rosalie

Diagramme de classes UML (classes principales)



Justification des choix OO

Abstraite

Une classe abstraite, c'est l'équivalent d'une base commune pour plusieurs classes. Elle rassemble ce qui est partagé, et oblige les sous-classes à gérer ce qui est spécifique.

Dans notre projet, la classe **Personne** est abstraite parce qu'un médecin et un patient sont avant tout des personnes : ils ont un nom, un prénom, une date de naissance en commun. Mais dans un hôpital, on ne crée pas juste "une personne", on crée un médecin ou un patient.

La classe **Soin** est abstraite aussi : une consultation et un acte chirurgical sont tous les deux des soins, avec un identifiant, une date et un type de soin. Mais un soin tout seul, sans type précis, ça n'a pas de sens métier.

Interfaces

Permet de spécifier un ensemble de méthodes obligatoires que toute classe implémentant doit fournir. L'avantage, c'est que des classes qui n'ont rien à voir entre elles peuvent partager un même comportement sans avoir besoin d'être liées par un héritage.

Dans notre projet, on a plusieurs interfaces, notamment **Soignable** : un patient peut recevoir des soins, un médecin peut en dispenser. À la place de coder ce comportement plusieurs fois, l'interface le définit une fois, et les deux classes n'ont plus qu'à l'implémenter.

Planifiable est semblable : un médecin et un infirmier ont tous les deux un planning. L'interface les oblige à gérer leurs disponibilités, mais chacun peut le faire à sa manière.

Facturable, lorsqu'un patient génère des frais à chaque soin, l'interface assure que tout ce qui peut être facturé expose les mêmes méthodes pour calculer le coût.

Et **Urgence**, c'est utile pour définir l'importance d'un soin, par exemple un acte chirurgical peut être urgent. L'interface permet de le mettre dans une file de priorité en fonction de l'urgence définie.

Les génériques

Permet d'écrire une classe ou une méthode qui peut bosser avec n'importe quel type, tout en gardant une bonne sécurité de typage..

Dans notre projet, on a une classe **Registre<T extends Entite>** qui nous permet de gérer une collection de patients, de médecins ou de soins avec le même code. Sans les génériques, on serait obligé de créer un **RegistrePatient**, un **RegistreMedecin**, cela nous empêche de faire des répétitions, mais avec des noms différents. Un code dupliqué pour une logique identique.

Collections

Chaque collection a son utilité. Cela permet d'éviter les bugs et le code devient plus clair et plus rapide.

Dans le projet, on utilise **List<Soin>** pour l'historique des soins d'un patient, mais aussi pour l'historique des soins donnés par un médecin.

On utilise d'autres listes comme **List<Lit>**, **List<LocalDateTime>**, **List<Patient>** qui permettent d'avoir des historiques sur d'autres données.

Map<Integer, Medecin> permet de trouver un médecin directement avec son id, sans fouiller toute la liste.

Streams et Lambdas

Les streams permettent de manipuler les collections de manière simple et lisible : filtrer, trier, transformer, sans avoir à écrire des boucles compliquées.

Dans le projet, pour trouver tous les patients hospitalisés d'un médecin donné, un `filter()` suffit en une ligne. Pour trier les soins par date, un `sorted()` avec un `Comparator` sur la date. Pour calculer le coût total des soins d'un patient, on enchaîne un `map()` et un `reduce()`. Sans streams, chaque opération demanderait plusieurs lignes de boucles.

Difficultés rencontrées et solutions adoptées

Lors du projet, nous avons rencontré plusieurs difficultés, notamment une difficulté concernant la transmission des données entre Servlet et JSP. La séparation claire MVC nous a permis de nous y retrouver plus facilement par rapport aux requêtes.

Par ailleurs, nous avons eu quelques difficultés au sujet de l'organisation du projet Git, particulièrement lors de l'assemblage du développement de chaque membre de l'équipe. Nous avons vérifié les commits avant de merge dans le dépôt final (main).

Notre plus grande difficulté a été le temps restreint que nous avions pour réaliser le projet, nous avons tout de même réalisé une répartition des tâches au sein de l'équipe et une communication régulière lors de l'avancement du projet.

Ce qui aurait été amélioré avec plus de temps

Avec plus de temps, nous aurions pu largement améliorer l'interface, son originalité et son expérience utilisateur, par manque de temps, elle est très simple. Nous aurions également pu améliorer la gestion des fonctionnalités pour les classes Métiers.